Chapter 1

Differences between Microprocessor and Microcontroller

Sl No	Microprocessor	Microcontroller
1	Microprocessors can be understood as the heart of a computer system.	Microcontrollers can be understood as the heart of an embedded system.
2	A microprocessor is a processor where the memory and I/O component are connected externally.	A microcontroller is a controlling device wherein the memory and I/O output component are present internally.
3	The circuit is complex due to external connection.	Microcontrollers are present on chip memory. The circuit is less complex.
4	The memory and I/O components are to be connected externally.	The memory and I/O components are available.
5	Microprocessors can't be used in compact system.	Microcontrollers can be used with a compact system.
6	Microprocessors are not efficient.	Microcontrollers are efficient.

7	**Microprocessors have less number of registers.**	**Microcontrollers have more number of registers.**
8	Microprocessors are generally used in personal computers.	Microcontrollers are generally used in washing machines, and air conditioners.
9	Cost is high	Cost is low
10	Do not have power saving mode.	Have a power-saving mode.
11	Uses an external bus.	Uses an internal controlling bus.
12	Can run at a very high speed.	Can run up to 200MHz or more.
13	Microprocessors are multitasking in nature. Can perform multiple tasks at a time. For example, on computer we can play music while writing text in text editor.	Single task oriented. For example, a washing machine is designed for washing clothes only.

ARM Embedded Systems

- ARM(Advanced RISC Machine) is a 32-bit architecture microcontroller that was developed by Acorn Computers in 1983.
- ARM is basically a family of Reduced Instruction Set Computing (RISC) architecture-based microprocessors. ARM microcontrollers consist of ARM processors, RAM, ROM, and I/O peripherals. ARM microcontrollers are used in a wide range of applications due to their low power consumption, low cost, and high performance.
- One of the important features of ARM microcontrollers is that they are highly customizable depending on requirements of the applications. Therefore, it is highly versatile microcontroller architecture.

ARM Embedded Systems

- We can use assembly language as well as high level programming languages such as C, C++ to program the ARM microcontrollers. ARM microcontrollers are highly scalable;
- Some popular microcontrollers of ARM family are ARM Cortex-Mo to M7, LPC2148, etc.
- The communication protocols used in ARM microcontrollers are UART, USART, SPI, I2C, I2S, LIN, CAN, DSP, SAI, and IrDA.
- ARM microcontrollers have faster clock speed, allowing them to process more instructions per second.

The RISC design philosophy

- The ARM core uses a RISC architecture.
- RISC is a design philosophy aimed at delivering simple but powerful instructions that execute within a single cycle at a high clock speed.
- The RISC philosophy concentrates on reducing the complexity of instructions performed by the hardware because it is easier to provide greater flexibility and intelligence in software rather than hardware.
- As a result, a RISC design places greater demands on the compiler.

- In contrast, the traditional complex instruction set computer (CISC) relies more on the hardware for instruction functionality, and consequently the CISC instructions are more complicated.

Differences between RISC and CISC

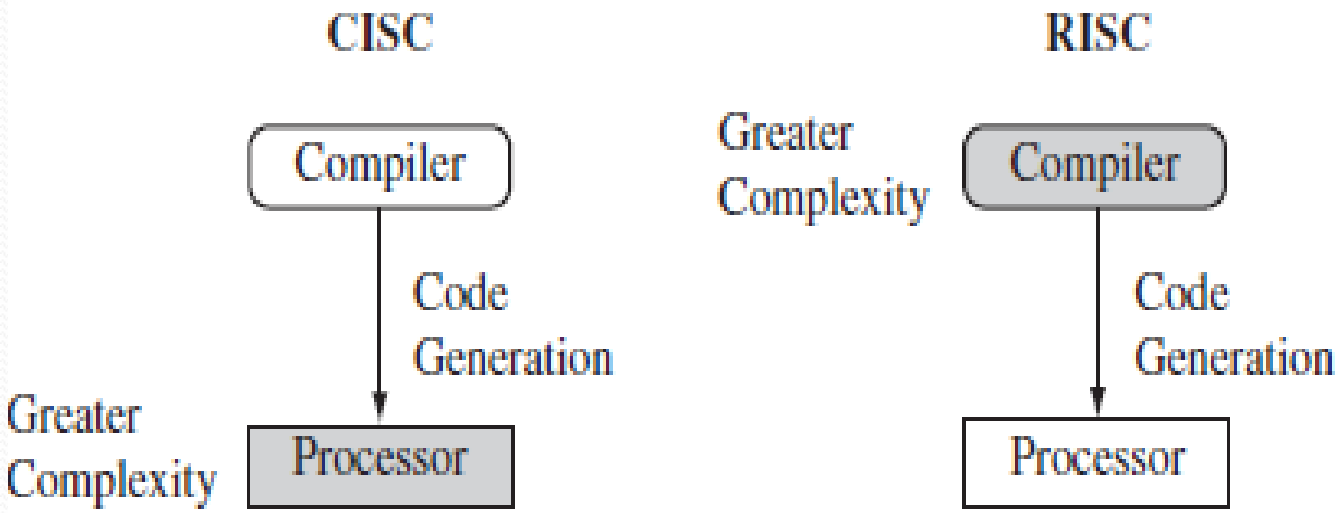
Sl No	CISC	RISC
1	CISC is the short form for Complex Instruction Set Computer.	RISC refers to Reduced Instruction Set Computer.
2	Instruction format is variable	Instruction format is Fixed
3	The CISC architecture processes complex instructions that require several clock cycles for execution. On average, it takes two to five clock cycles per instruction (CPI).	The RISC architecture executes simple yet optimized instructions in a single clock cycle. It processes instructions at an average speed of 1.5 clock cycles per instruction (CPI).
4	CISC focuses on hardware, such as transistors, to execute instructions.	RISC focuses more on software such as codes or compilers to execute instructions.
5	CISC uses a variety of instructions to accomplish complex tasks.	RISC is provided with a reduced instruction set, which is typically primitive in nature.
6	A CISC processor works with 16 bits to 64 bits to execute each instruction.	A RISC processor utilizes 32 bits to execute each instruction.
7	A CISC architecture uses one cache to store data as well as instructions. However, recent CISC designs employ split caches to divide data and instructions.	The RISC architecture relies on split caches, one for data and the other for instructions.
8	CISC processors use a memory-to-memory framework to execute instructions such as ADD, LOAD, and even STORE.	RISC processors rely on a register-to-register mechanism to execute ADD, STORE, and independent LOAD instructions.

Differences between RISC and CISC contd..

9	**The CISC architecture uses only one register set.**	**The RISC architecture utilizes multiple register sets.**
10	Since CISC devices operate in a multi-clock environment, it supports addressing modes in the range of 12 to 24.	Since RISC machines operate on single clock cycles, it has limited addressing modes.
11	CISC processors are capable of processing high-level programming language statements more efficiently	Since RISC processors support a limited set of addressing modes, complex instructions are synthesized through software codes.
12	CISC does not support parallelism and pipelining. As such, CISC instructions are less pipelined.	RISC processors support instruction pipelining.
13	CISC instructions require high execution time.	RISC instructions require less time for execution.
14	Some examples of CISC processors include Intel x86 CPUs, Motorola 68000 family,AMD(Advanced Micro Devices), 8086 family, 8051.	Examples of RISC processors include ARM(Advance Risc machine), MIPS(Microprocessor without Interlocked Pipeline Stages), SPARC(Scalable Processor Architecture), Power, RISC-V
15	CISC processors are used for low-end applications such as home automation devices, security systems, etc.	RISC processors are suitable for high-end applications, including image and video processing, telecommunications, etc.
16	ADD AX, BX	ADD R3, R1, R2

17	MOV AX, A ; Load A into AX
	ADD AX, B ; AX = AX + B (memory operand)
	MOV C, AX ; Store result into C
		LDR R1, A ; Load A into register R1
		LDR R2, B ; Load B into register R2
		ADD R3, R1, R2 ; R3 = R1 + R2
		STR R3, C ; Store result into C

RISC and CISC Major Differences



CISC vs. RISC. CISC emphasizes hardware complexity. RISC emphasizes compiler complexity.

The RISC philosophy is implemented with four major design rules:

1. ***Instructions—***

- *RISC processors have a reduced number of instruction classes.*
- *These* classes provide simple operations that can each execute in a single cycle.
- The compiler or programmer synthesizes complicated operations (for example, a divide operation) by combining several simple instructions.
- Each instruction is a fixed length to allow the pipeline to fetch future instructions before decoding the current instruction.
- In contrast, in CISC processors the instructions are often of variable size and take many cycles to execute.

2. ***Pipelines—***

- *The processing of instructions is broken down into smaller units that can be* executed in parallel by pipelines.
- Ideally the pipeline advances by one step on each cycle for maximum throughput.
- Instructions can be decoded in one pipeline stage.
- There is no need for an instruction to be executed by a mini-program called microcode as on CISC processors.

3.

3. ***Registers—***

- RISC machines have a large general-purpose register set.
- Any register can contain either data or an address. Registers act as the fast local memory store for all data processing operations.
- In contrast, CISC processors have dedicated registers for specific purposes.

4. *Load-store architecture:*

- *The processor operates on data held in registers.*
- *Separate load* and store instructions transfer data between the register bank and external memory.
- Memory accesses are costly, so separating memory accesses from data processing provides an advantage because you can use data items held in the register bank multiple times without needing multiple memory accesses.
- In contrast, with a CISC design the data processing operations can act on memory directly.

- These design rules allow a RISC processor to be simpler, and thus the core can operate at higher clock frequencies.
- In contrast, traditional CISC processors are more complex and operate at lower clock frequencies.

The ARM Design Philosophy

- There are a number of physical features that have driven the ARM (Advanced RISC Machines) processor design.
- First, portable embedded systems require some form of battery power.
- The ARM processor has been specifically designed to be small to reduce power consumption and extend battery operation—essential for applications such as mobile phones and personal digital assistants (PDAs).

The ARM Design Philosophy

- High code density is another major requirement since embedded systems have limited memory due to cost and/or physical size restrictions.

- High code density is useful for applications that have limited on-board memory, such as mobile phones and mass storage devices.

- ARM has incorporated hardware debug technology within the processor so that software engineers can view what is happening while the processor is executing code.

- With greater visibility, software engineers can resolve issues faster, which has a direct effect on the time to market and reduces overall development costs.

- The ARM core is not a pure RISC architecture because of the constraints of its primary application—the embedded system.

- The strength of the ARM core is that it does not take the RISC concept too far.

- In today's systems the key is not raw processor speed but total effective system performance and power consumption.

Instruction Set for Embedded Systems

- The ARM instruction set differs from the pure RISC definition in several ways that make the ARM instruction set suitable for embedded applications:

- *Variable cycle execution for certain instructions—*
- *Not every ARM instruction executes* in a single cycle.
- For example, load-store-multiple instructions vary in the number of execution cycles depending upon the number of registers being transferred.
- The transfer can occur on sequential memory addresses, which increases performance since sequential memory accesses are often faster than random accesses.
- Code density is also improved since multiple register transfers are common operations at the start and end of functions.

- *Inline barrel shifter leading to more complex instructions—The inline barrel shifter is* a hardware component that pre-processes one of the input registers before it is used by an instruction. This expands the capability of many instructions to improve core performance and code density.

- *Thumb 16-bit instruction set—ARM enhanced the processor core by adding a second* 16-bit instruction set called Thumb that permits the ARM core to execute either 16- or 32-bit instructions. The 16-bit instructions improve code density by about 30% over 32-bit fixed-length instructions.

Thumb 16-bit:

- The Thumb instruction set is a subset of the most commonly used 32-bit ARM instructions.
- 16-bit Thumb is a set of instructions in computer programming that are half the length of standard ARM instructions. They are used to reduce the size and cost of code storage memory in ARM processors.
- Thumb instructions are each 16 bits long, and have a corresponding 32-bit ARM instruction that has the same effect on the processor model.

- Thumb instructions operate with the standard ARM register configuration, allowing excellent interoperability between ARM and Thumb states.
- Thumb, therefore, makes the ARM7TDMI core ideally suited to embedded applications with restricted memory bandwidth, where code density is important.
- The availability of both 16-bit Thumb and 32-bit ARM instruction sets gives designers the flexibility to emphasize performance or code size on a subroutine level, according to the requirements of their applications.

- For example, critical loops for applications such as fast interrupts and DSP algorithms can be coded using the full ARM instruction set then linked with Thumb code.

Thumb-2:

Thumb-2 technology, introduced in ARMv6T2, extended the original Thumb instruction set with 32-bit instructions, enabling it to achieve performance similar to ARM code while maintaining improved code density.

- *Conditional execution—An instruction is only executed when a specific condition has* been satisfied. This feature improves performance and code density by reducing branch instructions.

- *Enhanced instructions—The enhanced digital signal processor (DSP) instructions were* added to the standard ARM instruction set to support fast 16×16-bit multiplier operations and saturation. These instructions allow a faster-performing ARM processor in some cases to replace the traditional combinations of a processor plus a DSP.

- These additional features have made the ARM processor one of the most commonly used 32-bit embedded processor cores.